

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Computer Science and Engineering
B.Tech II Semester-III
CS231 – Object Oriented Programming Practical Assignment
Lab Assignment - 8

1. Create a class **Number** that stores a single integer.
 - Overload the **pre-increment** (**++obj**) and **post-increment** (**obj++**) operators using **member functions**.
 - Pre-increment should increase the value first, then return it.
 - Post-increment should return the old value first, then increase it.
 - Write a **main()** function to show the difference between pre-increment and post-increment on the same object.
2. Your task is to create a **Point** class that represents a coordinate in a 2D plane.
 - Define a class **Point** with two private integer data members: **x** and **y**.
 - Implement a parameterized constructor to initialize a point with given x and y values.
 - **Unary Operator Overloading:** Overload the **pre-increment** (**++**) and **post-increment** (**++**) operators as **member functions**. Both operators should increment both the **x** and **y** coordinates by one. The **pre-increment** operator should return the updated object, while the **post-increment** operator should return the object's value **before** it was incremented.
 - **Binary Operator Overloading:** Overload the addition (**+**) operator as a **member function** to add two **Point** objects. The result should be a new **Point** object where the x-coordinates are added together, and the y-coordinates are added together.
 - Write a **display()** member function to print the point's coordinates in the format (x, y).
 - In your **main()** function, create **Point** objects and use the overloaded operators to demonstrate how they work. Print the results clearly to show the before and after states.
3. Create a class **Distance** with two integer data members: feet and inches.
 - Write a constructor to initialize the values.
 - Overload the **+** operator as a **friend function** to add two **Distance** objects.
 - If inches ≥ 12 , convert to feet.
 - Write a **display()** function to show the result in the form: X feet Y inches.
 - In **main()**, create two **Distance** objects, add them, and display the result.
4. Create a class **Complex** that represents complex numbers.
 - Define two private data members: **real** and **imag**.
 - Implement a parameterized constructor to initialize the values.
 - Overload the **binary + operator** as a **member function** to add two **Complex** numbers.
 - Write a **display()** function to print the result in the form **a + bi**.
 - In **main()**, create two **Complex** objects, add them using **+**, and display the result.
5. Create a class **Time** with two integer data members: hours and minutes.
 - Write a constructor to initialize the values.
 - Overload the **+** operator as a friend function to add two **Time** objects.
 - If minutes ≥ 60 , convert the extra minutes into hours (normalization).
 - Write a **display()** function to show the result in the form: X hours Y minutes.
 - In **main()**, create two **Time** objects, add them, and display the result.
6. Implement a Graphics library utility to perform operations on 2d vector data. Define a class to represent a 2D Vector that performs common mathematical operations with familiar operation syntax.
 - a. Initialize a 2D vector with some default values of x and y.

- b. Perform common arithmetic operations on 2d vector objects and return the result of the operation as a new 2d vector object.
 - i. Addition: $O1(x1, y1), O2(x2, y2) \rightarrow O3(x1+x2, y1+y2)$
 - ii. Subtraction: $O1(x1, y1), O2(x2, y2) \rightarrow O3(x1-x2, y1-y2)$
 - iii. Scaler Multiplication: $O1(x, y), \text{ scaler value}=c \rightarrow O2(x*c, y*c)$
 - iv. Dot Product: $O1(x1, y1), O2(x2, y2) \rightarrow (x1*x2) + (y1*y2) \rightarrow \text{return double value}$
 - v. Comparator (==): Compare the two 2d vector objects. If the values of the x and y components are the same in both objects, then it's equal.
- c. Display the result of the operation directly to the console output in format [x, y] by overloading the output stream (<<)